

hmCodeTrans: Human–Machine Interactive Code Translation

Jiaqi Liu , Fengming Zhang , Xin Zhang , Zhiwen Yu , *Senior Member, IEEE*,
Liang Wang , *Member, IEEE*, Yao Zhang , and Bin Guo , *Senior Member, IEEE*

Abstract—Code translation, i.e., translating one kind of code language to another, plays an important role in scenarios such as application modernization and multi-language versions of applications on different platforms. Even the most advanced machine-based code translation methods can not guarantee an error-free result. Therefore, the participation of software engineer is necessary. Considering both accuracy and efficiency, it is suggested to work in a human-machine collaborative way. However, in many realistic scenarios, human and machine collaborate ineffectively - model translates first and then human makes further editing, without any interaction. To solve this problem, we propose hmCodeTrans, a novel method that achieves code translation in an *interactive human-machine collaborative way*. It can (1) save the human effort by introducing two novel human-machine collaboration patterns: prefix-based and segment-based ones, which feed the software engineer's sequential or scattered editing back to model and thus enabling the model to make a better retranslation; (2) reduce the response time based on two proposed modules: attention cache module that avoids duplicate prefix inference with cached attention information, and suffix splicing module that reduces invalid suffix inference by splicing a predefined suffix. The experiments are conducted on two real datasets. Results show that compared with the baselines, our approach can effectively save the human effort and reduce the response time. Last but not least, a user study involving five real software engineers is given, which validates that the proposed approach owns the lowest human effort and shows the users' satisfaction towards the approach.

Index Terms—Code translation, human-machine collaboration, interactive translation.

I. INTRODUCTION

IN the field of intelligence technology, programming languages evolve much faster than natural languages, which gives rise to a significant demand of code translation. Code translation [1], [2], [3] aims to rewrite the existing code from one kind of programming language into another, e.g., from Python to Java. An effective program translation method could help companies in many practical development scenarios, such as: 1) Application modernization. For example, the Commonwealth Bank of Australia spent five years and \$750 million to convert its platform from Common Business Oriented Language (COBOL) to Java. 2) Multi-language versions of applications on different platforms. For example, when enterprises plan to apply Windows-based desktop programs to the Android platform, they may need to rewrite the C++ programs into Java.

The manual-based program translation methods are expensive and inefficient since it requires experts with sophisticated bilingual knowledge [4], [5], [6], [7]. Except for the purely manual way, the explosive growth of the amount of code data has facilitated machine learning-based program translation research. This kind of method owns low cost and high efficiency, which, however, is *not error-free* [8], [9], [10]. For example, TransCoder [8] reaches up to only 74.8% computational accuracy. When applying such methods in realistic tasks, it still requires human to check and edit the machine's result, which is called human post-editing. Therefore, in order to obtain an error-free translation result, both human (software engineer) and machine (machine learning model) are required to participate. *So, is there a more effective way for human-machine collaboration in code translation?*

In order to deal with this tricky problem, we propose to adopt the Human-Machine Computing (HMC) [11], [12], [13] framework into code translation. The core idea of HMC is to take the strengths of both human and machine so that the overall quality of code translation, that is, accuracy and efficiency, can be improved. An essential issue to improve human-machine collaboration is *interaction between software engineer and translation model* - first, translation model generates preliminary translation result to software engineer; then, the software engineer checks part of the result, makes editing and feeds it back to the translation model, based on which translation model makes retranslation and gives it to software engineer for further editing; repeat until all the errors have been corrected. As shown in Fig. 1, thanks to the interaction between software engineer

Manuscript received 18 October 2023; revised 7 March 2024; accepted 11 March 2024. Date of publication 20 March 2024; date of current version 16 May 2024. This work was supported in part by the National Science and Technology Major Project under Grant 2021ZD0113305, in part by the National Natural Science Foundation of China under Grant 61960206008 and Grant 62372381, and in part by the National Science Fund for Distinguished Young Scholars under Grant 62025205. Recommended for acceptance by F. Ferrucci. (Corresponding authors: Zhiwen Yu; Jiaqi Liu.)

Jiaqi Liu, Fengming Zhang, Xin Zhang, Liang Wang, Yao Zhang, and Bin Guo are with the School of Computer Science, Northwestern Polytechnical University, Xi'an 710129, China (e-mail: jqliu@nwpu.edu.cn; fmzhang@mail.nwpu.edu.cn; 2020262928@mail.nwpu.edu.cn; liangwang@nwpu.edu.cn; yaozh.g@nwpu.edu.cn; guob@nwpu.edu.cn).

Zhiwen Yu is with Haerbin Engineering University, Harbin 150001, China and also with the School of Computer Science, Northwestern Polytechnical University, Xi'an 710129, China (e-mail: zhiwenyu@nwpu.edu.cn)

Digital Object Identifier 10.1109/TSE.2024.3379583

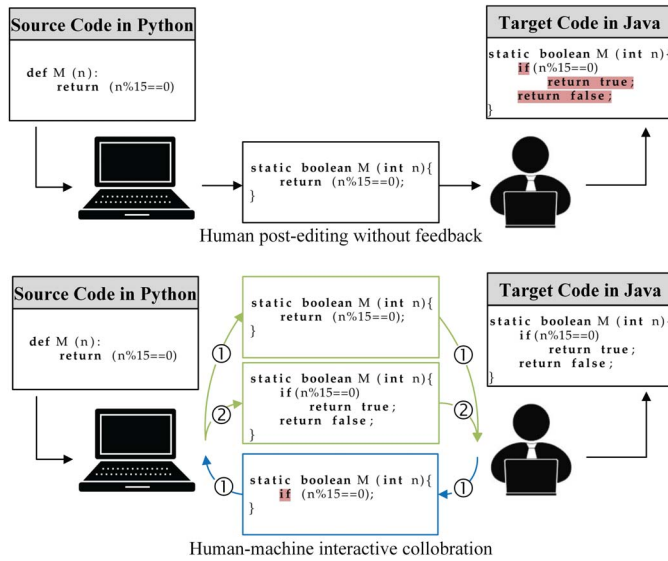


Fig. 1. Comparison between human post-editing method and human-machine interactive collaboration method. The number of tokens corrected by the software engineer is 7 in the former method but 1 in the latter method (colored with red).

and translation model, HMC-based method saves 85.7% human effort compared to human post-editing. Interactive translation has shown its benefits in natural language translation [14], [15], [16], [17]. However, to the best of our knowledge, there is no research that applies interactive translation to code language. When introducing interaction to code translation, there exist two challenges. **First**, due to the preciousness of human effort, *how to design effective interactive patterns that can reduce human workload?* **Second**, with large deep neural networks like transformer model [18], [19], [20], code translation inevitably involves high response time during the subsequent iterations and seriously affects the user's collaboration experience. So, *how to optimize the translation model in order to reduce response time?*

To save human effort, we propose to feed the editing made by software engineer back to the model and accomplish the translation task through interactive collaboration. Specifically, we introduce two human-machine collaboration patterns: prefix-based and segment-based ones. (1) Prefix-based pattern requires the software engineer to edit sequentially. In each round, the software engineer checks the newly generated tokens and stops once he/she finds a wrong one. Then, the software engineer corrects this wrong token. The confirmed/corrected tokens lay not behind the editing position and thus we call them prefix. (2) Segment-based pattern requires the software engineer to edit scatteredly. In each round, the software engineer checks all the tokens and only edits the first incorrect one. These confirmed/corrected tokens are scattered in the codes and form several segments. The above two patterns can greatly save human effort compared with human post-editing translation. They have their own advantages and disadvantages, which we will discuss in the following sections.

To reduce response time, we propose two modules to avoid redundant calculations during the model inference.

(1) Attention cache module is proposed to skip the inference of confirmed tokens. It stores the state information of the transformer architecture, i.e., key matrix and value matrix, generated in the previous round, based on which the model can skip the inference of the tokens that have been confirmed by the software engineer and directly start inferring from the unconfirmed tokens, which therefore reduces the execution time. (2) Suffix splicing module is proposed to skip the inference of tokens that will not be checked by the software engineer in the next round, called suffix. Since the software engineer stops checking once he/she makes a correction, the tokens behind editing position are meaningless to some extent. So we propose suffix splicing module to skip the inference of these tokens but splice an approximately same sequence to ensure the completeness of the codes.

In this paper, we propose hmCodeTrans - a human-machine interactive code translation method. The method's superiority lies in that comparing with the machine learning translation methods, it can greatly improve the translation accuracy and guarantee an error-free result, while comparing with some simple human-machine translation methods, e.g., code repair algorithms with human feedback, it can effectively save the human effort. It works in an interactive way. At the beginning, the model gives an initial translation result. Then in each round, the software engineer checks part of the result provided by the model, corrects only one wrong token he/she finds and feeds the confirmed tokens back to the model in the form of prefix or segments. Given the feedback, the model makes retranslation, in which it avoids duplicate prefix inference based on the attention information cached in the translation of the previous round, as well as reduces the invalid suffix inference by aborting early at the appropriate time step. To summarize, we made the following contributions:

- In order to save human effort, we propose two interactive human (software engineer) machine (translation model) collaboration patterns. To the best of our knowledge, this is the first attempt to introduce the natural language interactive translation approach to the code translation task.
- In order to reduce response time, we propose attention cache module and suffix splicing module to avoid duplicate inference of confirmed prefix and invalid suffix respectively, which can greatly improve the algorithm efficiency and the user satisfactory.
- We conduct extensive experiments on two realistic datasets. The results show that our approach saves human effort by up to 65.0% compared with human post-editing method, and reduces response time by up to 90.9% compared with interactive translation without the proposed modules.

II. RELATED WORK

This work introduces the interactive translation method in natural language to code translation, and thus we focus on two relevant fields: code translation as well as interactive translation method in natural language.

A. Code Translation

There exist four main categories of code translation methods: rule-based translation methods, supervised translation methods, unsupervised translation methods and methods combining pre-training on large monolingual corpora with fine-tuning on small bilingual corpora.

Rule-based translation methods: To meet some specific requirements, rule-based translation methods have been developed. Alsubhi et al. [4] developed a tool that translates sequential code to parallel code written in C++ and OpenACC. An et al. [5] gave an automatic inference of Java-to-Swift translation rules for porting mobile application. Pino et al. [6] explored the translation method of OpenMP to OpenACC 2.5. Tauda et al. [7] developed a codeming language translator for integration client application with web Application Programming Interfaces (APIs). These methods perform well, which, however, can only deal with a specific case but lack the generality.

Supervised translation methods: Earlier code translation methods are mainly supervised, whose training relies on massively parallel corpora. At first, some statistical code translation models are used: Karaivanov et al. [21] proposed a phrase-based statistical machine translation approach. Nguyen et al. [22] investigated how well statistical machine translation models for natural languages could help in code translation. Nguyen et al. [23] proposed a divide-and-conquer technique to make an improvement for phrase-based methods. Later, with the development of deep learning, research on code translation based on Tree to Tree model emerged. Chen et al. [10] used the attention mechanism to locate the corresponding subtree in the source tree and used the information of the subtree to guide the non-terminal extension. The main disadvantage of supervised translation methods is that they rely on high-quality training data.

Unsupervised translation methods: Thanks to a large amount of monolingual source code in code hosting platforms such as GitHub and Gitee, unsupervised translation methods emerged. Roziere et al. [8] proposed TransCoder, which trains a completely unsupervised code translation model on the source code of the GitHub project. Transformer language model, e.g., Palm [24], has shown its advance in various applications, including not only natural languages, but also code languages. What's more, some additional approaches are used to improve the reliability of neural machine translation. Szafraniec et al. [25] leveraged low-level compiler intermediate representations, through which the number of correct translations generated by TransCoder for C++, Java, Go and Rust increases by 11%.

Pre-training and fine-tuning methods: Since we can get large-scale monolingual source corpus and small-scale bilingual parallel corpus, a better solution is to pre-train the model on the monolingual set and then fine-tune the downstream tasks in the parallel corpus. Roziere et al. [9] proposed DOBF that leverages the structural aspect of codeming languages and pre-trains a model to recover the original version of obfuscated source code. Wang et al. [26] proposed three pre-training tasks based on the T5 model: span prediction, identifier prediction, and identifier tagging. Feng et al. [27] used a bert-like target

preprocessing task in an open-source GitHub repository, and then many relevant research [28], [29] emerged. Roziere et al. [30] proposed to leverage an automated unit-testing system to filter out invalid translations in programming languages, thereby creating a fully tested parallel corpus.

The above four categories greatly promote the development of code translation. However, all these methods are not error-free, e.g., including compilation or logical errors, and therefore they require human post-editing, i.e., software engineers edit the results generated by the model to ensure the correctness. Some research have explored the more effective cooperation between human (software engineer) and machine (translation model). Weisz et al. [31], [32] proposed a human-computer collaboration scheme named Automatic Downstream Code Editing (ADCE). It introduces confidence highlighting and alternate translations to assist engineers in the code translation process, which, however, is quite simple and can not take full advantages of human and machine.

B. Interactive Translation Method in Natural Language

Our proposed approach is inspired by the interactive translation method in natural language. Foster [33] first introduced interactive-predictive machine translation for natural language, which investigates how human feedback acts on subsequent machine translation in a continuous iterative translation. Following it, many subsequent interactive translation methods are proposed. Peris et al. [14] implemented prefix-based protocols in a neural machine translation system and proposed a fragment-based collaborative approach. Later, it extended the granularity of the prefix-based collaborative approach from word level to character level and introduced the online learning approach to interactive translation systems [15]. Gupta et al. [16] incorporated lexical syntactic descriptions into the Transformer model in the form of combinatory categorial grammar supertags and implemented prefix-based interactive protocols with the Transformer model. Heer et al. [17] demonstrated that designing artificial intelligence into interactive systems can help to solve many complex tasks, including translation. It investigates a predictive interactive translation approach to let machines automate rote tasks while human professionals provide nuanced guidance. In addition, Zhao et al. [34] studied how to balance translation quality and the cost of human involvement. Peris et al. [35] studied the application of active learning to the interactive translation, whose main idea is to select, from an unbounded stream of source sentences, those worth to be supervised by a human agent.

The human-machine collaboration in the interactive manner is more effective compared to post-editing, since the machine can retranslate based on the software engineer's feedback. However, to our best knowledge, this is the first attempt to introduce human-machine interactive translation into the code translation.

III. INTERACTIVE TRANSLATION

In this section, we first provide a review on interactive translation in natural language, and then introduce how to apply it to code translation.

TABLE I
NOTATION AND DEFINITION

Notation	Definition
X	Code sequence of source code language.
J	Length of code sequence X .
Y	Code sequence of target code language.
I	Length of code sequence Y .
Y^t	Code sequence of target code language in the t -th iteration.
I_t	Length of code sequence Y^t .
y_i^t	The i -th token in Y^t .
$y_{c_t}^t$	Token that the software engineer corrects in the t iteration.
$y_{s_t}^t$	Token behind which the suffix code is spliced.
S_i^t	State information of token y_i^t , containing K_i^t and V_i^t .
A_i^t	Dictionary of token y_i^t .
F^t	Set of human confirmed segments in iteration t .
f_i^t	The i -th segment in F^t .
M^t	Set of machine generated segments in iteration t .
m_i^t	The i -th segment in M^t .

Interactive translation [33] requires human and machine to translate interactively in token granularity. In what follows, we study the translation between high-level programming languages like Python and Java, in which code is presented as a sequence of token - the basic syntax unit in programming language. Before illustrating on details of interactive code translation, we first introduce some mathematical notations that will be used later. We denote the source code token sequence as $X = \{x_1, \dots, x_J\}$, where J is the length of X , and x_i is the i -th token of X . We denote the translation result in the t -th iteration as $Y^t = \{y_1^t, \dots, y_{I_t}^t\}$ where I_t is the length of Y^t that varies in each iteration. The initial translation sequence is denoted as $Y^0 = \{y_1^0, \dots, y_{I_0}^0\}$ and the final sequence is denoted as $Y = \{y_1, \dots, y_I\}$. The notations and their definitions are presented in Table I.

Interactive translation first lets the model make translation on the source code X and obtains the initial result Y^0 , then software engineer and translation model iterate collaboratively. As shown in Fig. 2, in the t -th iteration, (i) the software engineer continues checking the rest of the translation sequence generated in the previous round after the last correction position (checks from the beginning if this is the first round), and corrects the first wrong token, denoted by $y_{c_t}^t$. The subscript “c” indicates “correct” and c_t represents the correction in the t -th iteration. The correction can impact the regeneration of the tokens after the editing point, which will be described in details later. In addition, it indicates that he/she has confirmed the prefix of $y_{c_t}^t$, which results in some confirmed tokens $f = \{y_1^t, \dots, y_{c_t}^t\}$. (ii) Based on the software engineer’s feedback f , the model makes regeneration. The regenerated result is denoted as Y^t . This process can be mathematically described as below:

$$Y^t = \arg \max \Pr(Y^t | X, Y^{t-1}, f = \{y_1^t, \dots, y_{c_t}^t\}), \quad (1)$$

where $f = \{y_1^t, \dots, y_{c_t}^t\}$ denotes the feedback provided by the software engineer. Equation (1) can be re-wrote as

$$Y^t = \arg \max \prod_{i=1}^{I_t} \Pr(y_i^t | X, Y^{t-1}, f = \{y_1^t, \dots, y_{c_t}^t\}). \quad (2)$$

Applying transformer architecture [16], [19] to the interactive translation, $\Pr(y_i^t)$ can be obtained as

$$\Pr(y_i^t) = \bar{y}_i^t \text{softmax}(VA_i), \quad (3)$$

where $\bar{y}_i^t$ is the one-hot representation of y_i^t , $V \in \mathbb{R}^{v \times d}$ is a weight matrix of linear layer (v is the size of vocabulary table, and d is the dimension of hidden layer), A_i is the output of the decoder of the transformer model for the i -th token (dictionary of the i -th token), and $\text{softmax}(VA_i)$ is the output of the transformer based model.

The tokens received by the model can be divided into two types

- 1) Confirmed tokens $\{y_1^t, \dots, y_{c_t}^t\}$: the ones that have been confirmed by the software engineer, which are correct ones requiring no modifying. These tokens can be directly obtained from the feedback f .
- 2) Unconfirmed tokens $\{y_{c_t+1}^{t-1}, \dots, y_{I_{t-1}}^{t-1}\}$: the tokens that receive no confirmation, which may contain errors and thus requires regeneration. These tokens are the tokens in Y^{t-1} located behind $y_{c_t}^t$.

For the confirmed tokens $\{y_1^t, \dots, y_{c_t}^t\}$, we remain them unchanged through a constrained generation. For the unconfirmed tokens $\{y_{c_t+1}^{t-1}, \dots, y_{I_{t-1}}^{t-1}\}$, the model retranslates them as $\{y_{c_t+1}^t, \dots, y_{I_t}^t\}$ through a free generation, based on the confirmed prefix $\{y_1^t, \dots, y_{c_t}^t\}$. Therefore the probability of y_i^t in Equation (2) can be revised as

$$\begin{aligned} \Pr(y_i^t | X, Y^{t-1}, f = \{y_1^t, \dots, y_{c_t}^t\}) \\ = \begin{cases} \delta(y_i^t, y_{c_t}^{t-1}), & i \in [1, c_t] \text{ (confirmed token);} \\ \bar{y}_i^t \text{softmax}(VA_i), & i \in [c_t+1, I_t] \text{ (unconfirmed token).} \end{cases} \end{aligned} \quad (4)$$

Applying interactive translation to code translation can ensure the correctness of translation with limited human labor. However, directly applying the existing interactive method to code translation has a significant flaw - in each round, the model makes a full prediction, i.e., a constrained generation from the first token to the corrected one, and a free generation from the token behind the corrected one to the last one. It results in a long response time, e.g., about 9 seconds for code with a length over 200, which is unbearable for the software engineer who waits to provide the feedback.

In order to overcome the aforementioned flaw, we propose hmCodeTrans. According to the different interaction methods, it can be divided into two types: prefix-based hmCodeTrans and segment-based hmCodeTrans. Prefix-based pattern requires the software engineer to edit sequentially. In each round, the software engineer checks the newly generated tokens from the last correction position in the previous round, and stops once he/she finds a wrong one. Segment-based pattern requires the software engineer to edit scatteredly. In addition to the actions required in prefix-based pattern, the software engineer also checks the tokens behind the wrong one and feeds the correct ones back in the form of segments. More details on the two feedback patterns are given in Fig. 2. The details of these two methods are illustrated in Section IV and Section V.

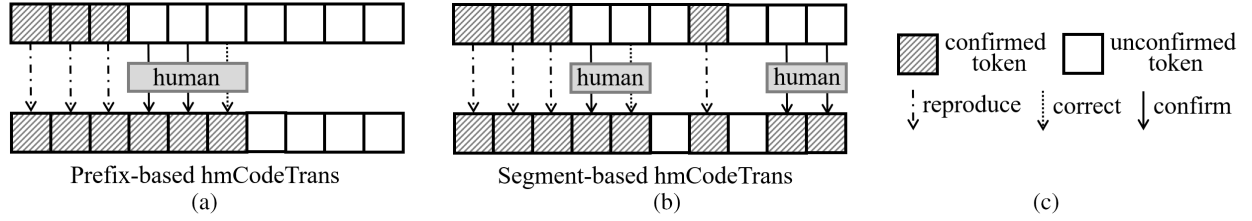


Fig. 2. How human works in prefix-based hmCodeTrans and segment-based hmCodeTrans. (i) Prefix-based hmCodeTrans: the software engineer checks the result generated in iteration $t - 1$ token by token, starting from the position behind last editing and ending once he/she finds a wrong token; (ii) segment-based hmCodeTrans: in addition to the confirmation in prefix, the software engineer is required to confirm the correct tokens located behind, which formed as segment.

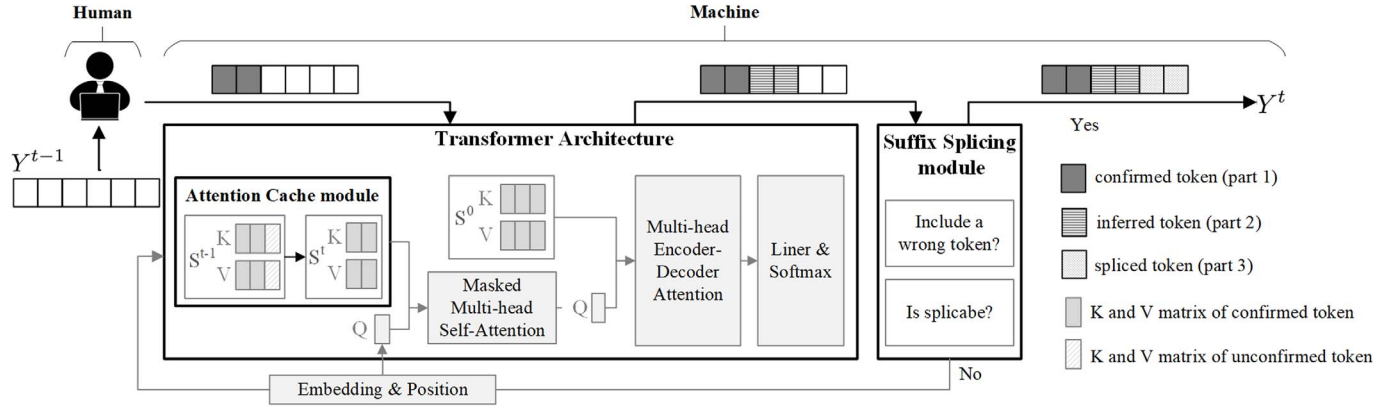


Fig. 3. Framework of prefix-based hmCodeTrans.

IV. PREFIX-BASED HMCODETRANS

This section first gives the intention and the overview of prefix-based hmCodeTrans in Section IV-A. Then it illustrates two important modules of the proposed method, i.e., attention cache module and suffix splicing module, in Section IV-B and Section IV-C.

A. Overview

The process described in Section III involves two parts of redundant calculation: tokens in prefix and tokens in suffix.

1) *Redundant Calculation of Tokens in Prefix*: Tokens in prefix $\{y_1^t, \dots, y_{c_t-1}^t\}$ are generated in the $(t - 1)$ -th iteration by model and confirmed as correct ones in the t -th iteration by human. The regeneration of them follows the constrained decoding as illustrated in Equation (4), which, however, is redundant and can be skipped with the proposed Attention Cache Module.

2) *Redundant Calculation of Tokens in Suffix*: As illustrated in Section III, in each iteration the model retranslates all the remaining tokens behind the corrected one, i.e., from $y_{c_t+1}^t$ to $y_{I_t}^t$. However, there exist some redundant calculations in this process. Specifically, in the next iteration, the software engineer will continue to confirm the retranslated results and stop after he/she finds a wrong token $y_{c_{t+1}}^{t+1}$. So, the tokens behind it can receive no feedback. Then, the model conducts regeneration, after which these tokens will be covered by the regenerated ones. Consequently, the tokens behind $y_{c_{t+1}}^{t+1}$ should be able to reflect the function of the remaining code so as to provide

a support to the software engineer, but are unnecessary to be exactly correct. As a result, we propose *Suffix Splicing Module* to make early termination in regeneration, which avoids redundant calculation by aborting the translation process early at an appropriate token $y_{s_t}^t$ (a prediction of $y_{c_{t+1}}^{t+1}$), and splicing a pre-defined suffix code sequence instead.

The translation result Y^t can be divided into three parts:

$$Y^t = \{y_1^t, \dots, y_{c_t}^t\} \oplus \{y_{c_t+1}^t, \dots, y_{s_t}^t\} \oplus \{y_{s_t+1}^t, \dots, y_{I_t}^t\}. \quad (5)$$

The objective of prefix-based hmCodeTrans is to avoid redundant calculation in tokens $\{y_1^t, \dots, y_{c_t}^t\}$ using the proposed Attention Cache Module, as well as redundant calculation to tokens $\{y_{s_t+1}^t, \dots, y_{I_t}^t\}$ using the proposed Suffix Splicing Module. The framework of Prefix-based hmCodeTrans is provided in Fig. 3. Specifically, the result in the last round Y^{t-1} is first given to the human for confirmation and then given to the model with transformer architecture for regeneration, in which attention cache module is used to avoid redundant calculation before $y_{c_t}^t$. Finally, suffix splicing module is used to avoid redundant calculation after $y_{s_t+1}^t$.

B. Attention Cache Module

In order to avoid redundant calculation in the first part of tokens $\{y_1^t, \dots, y_{c_t}^t\}$, we introduce Attention Cache module to the inference process.

In this module, we directly use tokens $\{y_1^{t-1}, \dots, y_{c_t-1}^{t-1}\}$, i.e., the tokens generated in the $(t - 1)$ -th iteration and confirmed in the t -th iteration, to replace tokens $\{y_1^t, \dots, y_{c_t-1}^t\}$. In addition,

we store the key matrix K and the value matrix V generated in the previous iteration, i.e., $K_{c_t-1}^{t-1}$ and $V_{c_t-1}^{t-1}$, using attention cache, and then use them to continue regenerating. Similarly as that in Equation (4), we can obtain $y_{c_t}^t$ through constrained retranslation and the tokens after $y_{c_t}^t$ through free retranslation. As a result, $\{y_{c_t+1}^t, \dots\}$ can be obtained as

$$\{y_{c_t+1}^t, \dots\} = \arg \max \Pr(\{y_{c_t+1}^t, \dots\} | X, S^0, S^{t-1}, f), \quad (6)$$

where

$$\begin{aligned} S^0 &= \{K^0, V^0\} \\ S^{t-1} &= \{K_{c_t-1}^{t-1}, V_{c_t-1}^{t-1}\}. \end{aligned}$$

S^0 and S^{t-1} are the state information in the first iteration and the previous iteration, respectively.

C. Suffix Splicing Module

In order to avoid redundant calculation in the third part of tokens $\{y_{s_t+1}^t, \dots, y_{I_t}^t\}$, we introduce Suffix Splicing module to the inference process. Specifically, we abort the translation process early at an appropriate token, called abort point, and splice a pre-defined suffix code sequence at the abort point. There are two issues to be explored: (1) how to obtain a pre-defined suffix code sequence that can be spliced at the abort point, and (2) how to find an appropriate abort point.

1) *How to Obtain a Pre-Defined Suffix Code Sequence?:* As illustrated, a pre-defined suffix code sequence should be able to reflect the function of the remaining code, but is unnecessary to be exactly correct. In Suffix Splicing module, we adopt the translation results in the first round, i.e., Y^0 , to be the suffix code sequence. By doing so, we calculate Y^0 only once at the beginning and therefore can avoid redundant calculation to a great extent. Meanwhile, Y^0 can reflect the function of the remaining code since it is translated from X .

2) *How to Choose an Appropriate Abort Point?:* Before the discussion on how to choose the abort point, we should first figure out what is an appropriate abort point.

The choice of the abort point is crucial. With an early abort point, the translated tokens may not include any wrong token, and thus the software engineer can not make any correction. It results in that the model can not fully use the software engineer's feedback, and thus the translation results receive no tuning in this round. However, with a late abort point, the translated sequence may include more than one wrong tokens. Since the software engineer will stop at the first wrong token, the remaining translation results have no use but result in a waste of calculation resource. As a result, the optimal abort point should be where the software engineer happens to correct next time. In addition, an appropriate abort point should be available to be spliced. Note in some cases, the token can not be spliced, e.g., y_i^t is a newly generated token that does exist in Y^0 .

Denote the abort point in the t -th iteration as s_t . We have

$$\{y_{s_t+1}^t, \dots, y_{I_t}^t\} = \{y_{s_0+1}^0, \dots, y_{I_0}^0\}.$$

Algorithm 1: Abort point searching and suffix splicing.

Input: y_i^t , the current token;

A_i^t , the dictionary probability distribution of y_i^t ;

Y^t , the current translated token sequence;

Y^0 , the initial translated token sequence;

A^0 , the dictionary probability distribution of Y^0 .

Output: $\{y_{s_0+1}^0, \dots, y_{I_0}^0\}$, the suffix to be spliced.

```

1 if classifier( $i - c_t, d, F_{vk}$ ) equals to 1 then
2   foreach  $y_j^0 \in Y^0$  do
3     if  $y_i^t$  equals to  $y_j^0$  and  $\text{consine}(A_i^t, A_j^0) > \delta$ 
4       then
5         return  $\{y_{s_0+1}^0, \dots, y_{I_0}^0\} = \{y_j^0, \dots, y_{I_0}^0\}$ 
5 else
6   return False

```

It is because we splice the initial translation results, i.e., Y^0 , as the suffix. Note the spliced sequence starts from the token $y_{s_0+1}^0$, where s_0^t is the position in Y^0 corresponding to s_t in the current iteration.

As a result, we choose the abort point in two steps: First, we judge whether the translation result $\{y_{c_t+1}^t, \dots, y_{s_t}^t\}$ has already contained a wrong token. Next, we evaluate whether this token sequence can be spliced with the suffix $\{y_{s_0+1}^0, \dots, y_{I_0}^0\}$. The details are shown in Algorithm 1.

For step 1 (line 1 in Algorithm 1), we use the probability distribution of y_i^t to judge whether the newly generated sequence has included a wrong token. Specifically, we use the normalized difference between the top k ($1 < k \leq K$) probability and the top 1 probability to measure the confidence of the model's result. This difference is defined as

$$d_i^k = \frac{P_i^{\text{top}k} - P_i^{\text{top}1}}{P_i^{\text{top}1}},$$

where $P_i^{\text{top}k}$, $1 < k < K$, denotes the value of the k -th largest probability for the token y_i in the current iteration.

We count the number of tokens whose difference d_i^k falls into a certain range as the feature

$$F_{vk} = |\{y_i | d_i^k < v\}|,$$

where

$$\begin{aligned} v &\in \{0.1, 0.2, \dots, 0.9\} \\ k &\in \{2, 3, \dots, K\}. \end{aligned}$$

The notation v is the value range of d_i^k , and we set it as the above equation in our implementation.

We adopt a classifier to judge whether the code sequence has contained a wrong token, with features F_{vk} , $i - c_t$ (length of current predicted sequence) and d (translation direction - from Python to Java or from Java to Python).

$$C = \text{classifier}(i - c_t, d, F_{vk}).$$

If $C = 1$, it indicates that the sequence has contained a wrong token, and therefore we can stop the translation. Otherwise, the translation process continues.

To obtain the training samples, the model compares the spliced suffix and the ground truth to check whether there exists a error in the decoding. If there exists a error, the data is a positive sample; otherwise, it is a negative sample. To evaluate how the type of classifier affects the results, we involve SVM, GBDT and AdaBoost classifiers in the experiment respectively. More details about the experimental results of the classifiers can be found in Table XIII.

For step 2 (line 3 in Algorithm 1), we judge whether the sequence $\{y_{c_t+1}^t, \dots, y_{s_t}^t\}$ can be spliced with the sequence $\{y_{s_0+1}^0, \dots, y_{I_0}^0\}$ by two conditions:

- 1) The last token of generated sequence should be equal to the token located just before the spliced sequence, i.e.,

$$y_{s_t}^t = y_{s_0}^0. \quad (7)$$

- 2) The cosine similarity between the translation model's output of the token $y_{s_t}^t$ and that of the token $y_{s_0}^0$ is large enough, i.e.,

$$\text{Cosine}(A_{s_t}^t, A_{s_0}^0) > \delta. \quad (8)$$

This is because the outputs of the same token in initial sequence and the t -th iteration sequence should have a high similarity. In the experiments, δ is set as 0.99. As used in Section III, A_i^t denotes the output of the token at the position i in the t -th iteration - the probability distribution over all the possible tokens in the dictionary. It belongs to a hash table created by the translation system.

In order to improve the efficiency, we run suffix splicing module with an increment 10, i.e., the model checks if the current position is spliceable every 10 tokens. The whole procedure of hmTransCode is given in Algorithm 2.

To summarize, in the prefix-based collaboration pattern, we use one model called TransformerPre() to generate the result of initial round as well as that of subsequent rounds. In the initial round, the input of the model is the source code sequence, and the output is the target code sequence. In the subsequent rounds, the input of the model consists of the source code sequence as well as the user feedback, and the output is the target code sequence with early abortion. The model is trained by fine-tuning the bilingual corpus of program translation using the pre-trained CodeT5 model. Last but not least, TransformerPre() can be replaced by any transformer-based model with the required input and output as illustrated above. The training of the alternative model follows the same way.

V. SEGMENT-BASED hmCODETRANS

This section first gives the intention and the overview of segment-based hmCodeTrans in Section V-A. Then it illustrates an important module of the proposed method, i.e., segment splicing module, in Section V-B.

A. Overview

In prefix-based hmCodeTrans, the software engineer is required to make confirmation sequentially, which results in a

Algorithm 2: Prefix-based hmCodeTrans.

Input: X , the source code sequence.

Output: Y , the target code sequence.

```

1 Obtain the initial translated code sequence  $Y^0$  from  $X$ .
2 Store  $Y^0$  and its state information  $S^0$ .
3 foreach iteration  $t$  do
4   Wait for the software engineer' correction and
   receive  $y_{c_t}^t$ .
5    $Y^t = \emptyset$ ;
6   foreach token  $y_i^t$  do
7     Part 1: Generate the tokens  $\{y_1^t, \dots, y_{c_t}^t\}$ .
8     if  $i \leq c_t$  then
9        $y_i^t = y_i^{t-1}$ ;
10       $Y^t = Y^t \oplus y_i^t$ .
11     Part 2: Generate the tokens  $\{y_{c_t+1}^t, \dots, y_{s_t}^t\}$ .
12     else
13        $[A_i^t, S_i^t] =$ 
14         Transformer( $X, S^0, S^{t-1}, \{y_1^t, \dots, y_{c_t}^t\}$ );
15        $y_i^t = \arg \max(A_i^t)$ ;
16        $Y^t = Y^t \oplus y_i^t$ .
17     Part 3: Generate the tokens  $\{y_{s_t+1}^t, \dots, y_{I_t}^t\}$ .
18     if  $i \% \text{increment equals to } 0$  then
19       if  $y_i^t$  is an abort point then
20         Get  $\{y_{s_0+1}^0, \dots, y_{I_0}^0\}$ ;
21          $Y^t = Y^t \oplus \{y_{s_0+1}^0, \dots, y_{I_0}^0\}$ .
22         return  $Y^t$  to the software engineer.
23         break.
```

limited range of user feedback. To enlarge the range of user feedback, we propose an alternative approach - segment-based hmCodeTrans. **Segment** refers to a continuous token sequence with some same features. As shown in Fig. 2, in each iteration, the approach requires the software engineer to explicitly pick up all the correct tokens (formed as several segments) in the current result Y^t , feeds them, along with the confirmed prefix and the corrected token, back and updates the results. Repeat until all the tokens have been confirmed or corrected by the software engineer.

Specifically, we re-define f in Equation (1) as

$$f' = \{\{y_1^t, \dots, y_{c_t}^t\}, F^t\},$$

where

$$F^t = \{f_1^t, \dots, f_{N_t}^t\}.$$

F^t represents the set of segments confirmed by the software engineer in iteration t , f_n^t is the n -th confirmed segment, and N_t is the number of confirmed segments, which varies in each iteration. In order to ensure an unified formation, we additionally take code terminator, i.e., the last token in segment $f_{N_t}^t$, into consideration. If the intermediate translation result ends with a confirmed segment, $f_{N_t}^t$ contains the confirmed tokens as well as the terminator; otherwise, $f_{N_t}^t$ contains only the terminator.

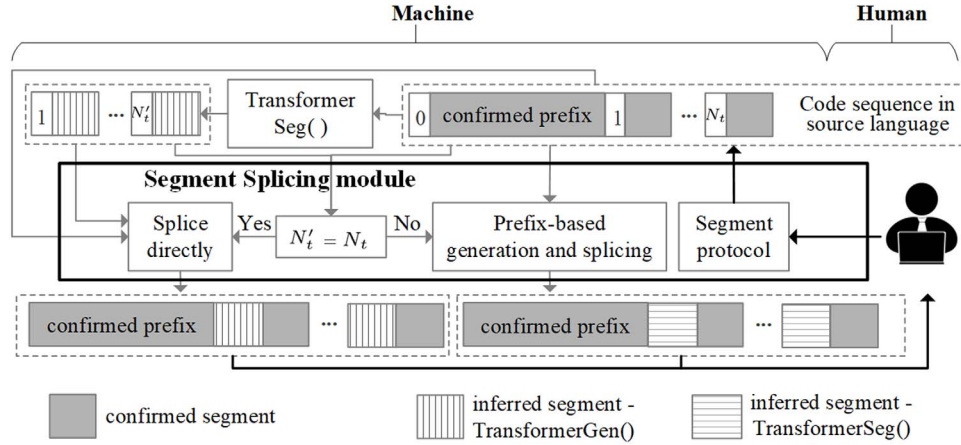


Fig. 4. Framework of segment-based hmCodeTrans.

Obviously, with the feedback f' , the translation model focuses on how to generate a group of segments, denoted by M^t , that can be spliced with the confirmed ones F^t .

$$M^t = \{m_1^t, \dots, m_{N_t'}^t\}.$$

When splice the two sets of segments, m_n^t should be located between the segment f_{n-1}^t and the segment f_n^t . Specially, m_1^t is filled behind the prefix $\{y_1^t, \dots, y_{c_t}^t\}$ and before the first confirmed segment f_1^t .

We introduce two translation models in segment-based hmCodeTrans. (1) TransformerGen(): a general translation model that is used to translate from X to Y^0 , whose input and output are both complete codes; (2) TransformerSeg(): a segment translation model that is used to translate from Y^{t-1} to Y^t , whose input is the complete source code and the confirmed prefix, the corrected token as well as a group of confirmed segments, and output is a group of translated segments to splice. Fig. 4 provides an overview of the framework of segment-based hmCodeTrans.

There are two significant problems that need to be solved: How to design a protocol to distinguish each segment? How to splice when the amount of confirmed segments is not equal to the amount of generated ones? We propose segment splicing module to deal with these two issues.

B. Segment Splicing Module

Segment splicing module is designed to splice the confirmed prefix as well as segments from the software engineer and the translated segments from the transformer model. It mainly solves the following two problems.

1) *How to Design a Segment Protocol?*: In order to better proceed the segments, we concat all of them as well as the prefix, form them as a sequence and introduce some identifiers to separate them. Inspired by Wang et al. [26], we regard the translation as a special sequence generation task. We use CodeT5 model to make pre-training, but add some special tokens (identifiers):

- Language-level separator $\langle \text{seq} \rangle$: identify the translation direction, e.g., from Python to Java or from Java to Python.

- Segment-level separator $\langle \text{segment_id_num} \rangle$: separate the segments, which is located between two segments.
- Placeholder $\langle \text{none_token} \rangle$: is used if the location has no token.

The input of the transformer model can be formulated as

$$\text{Input} = X \langle \text{seq} \rangle y_1^t \dots y_{c_t}^t \langle \text{segment_id_1} \rangle f_1^t \dots \langle \text{segment_id_N} \rangle f_{N_t}^t,$$

and the output of the transformer model is

$$\text{Output} = \langle \text{segment_id_1} \rangle m_1^t \dots \langle \text{segment_id_N} \rangle m_{N_t'}^t.$$

What's more, if there exists a segment m_i^t whose length is zero (the segments f_{i-1}^t and f_i^t can be spliced directly), we represent this segment by $\langle \text{none_token} \rangle$, which can help to remove some redundant tokens.

2) *How to Splice the Two Groups of Segments?*: The details of segment-based hmCodeTrans are given in Algorithm 3. In each iteration, TransformerSeg() outputs a group of inferred segments, and then we splice them with the confirmed segments. There are two possible cases:

- Case 1: The number of generated segments m_i^t is equal to that of confirmed segments f_i^t .
- Case 2: The number of generated segments m_i^t is not equal to that of confirmed segments f_i^t .

For case 1, we can splice the two parts of segments directly (lines 6 and 7 in Algorithm 3). However, for case 2, we can not do it (lines 8 and 9 in Algorithm 3). Therefore we introduce a Prefix-based Generation and Splicing method (PGS). It generates inferred tokens similar as in prefix-based hmCodeTrans, but splices segments rather than suffix.

The details of PGS are provided in Algorithm 4. It makes free translation for the unconfirmed tokens and constrained translation for the confirmed tokens.

$$\begin{aligned} \Pr(y_i^t | X, Y^{t-1}, f') &= \{\{y_1^t, \dots, y_{c_t}^t\}, F^t\} \\ &= \begin{cases} \delta(y_i^t, y_i^{t-1}), & y_i^{t-1} \text{ has been confirmed;} \\ y_i^t \text{ softmax}(VA_i), & \text{otherwise.} \end{cases} \end{aligned}$$

Algorithm 3: Segment-based hmCodeTrans.

Input: X , the source code sequence.
Output: Y , the target code sequence.

- 1 Obtain the initial translated code sequence Y^0 from X , through $\text{TransformerGen}()$;
- 2 Software engineer confirms and corrects, and store all the confirmed and corrected segments into F .
- 3 **while** *tokens are not totally confirmed* **do**
- 4 **foreach** *iteration* t **do**
- 5 $M^t = \text{TransformerSeg}(X, Y^{t-1}, f')$.
- 6 **if** *the length of F^t equals to that of M^t* **then**
- 7 $Y^t = \{y_1^t, \dots, y_{c_t}^t\} \oplus m_1^t \oplus f_1^t \oplus \dots \oplus m_{N_t}^t \oplus f_{N_t}^t$.
- 8 **else**
- 9 Make inference and splicing with PGS.
- 10 Software engineer confirms and corrects, and store all the confirmed and corrected segments into F .
- 11 $Y = Y^t$.

The details of Algorithm 4 is illustrated as below. Firstly, the method makes free translation with $\text{TransformerGen}()$ (lines 5 and 6 in Algorithm 4). Next, if the corresponding token in the previous iteration has been confirmed, in other words, belonging to the confirmed prefix $\{y_1^t, \dots, y_{c_t}^t\}$ or segments F^t , replace the generated token by the confirmed one (lines 7 to 10 in Algorithm 4). Otherwise, judge whether the current token y_i^t can be spliced with the next segment. If it does, update the current segment and make splicing (lines 11 to 14 in Algorithm 4). Finally, if the current token has not been confirmed and can not be spliced, retain it and concat it with the translated sequence (lines 15 and 16 in Algorithm 4). PGS results in low efficiency of case 2. Fortunately, in our experiments, case 2 happens not very often (about 4.1%) and thus has little effect on the final result.

To summarize, in the segment-based collaboration pattern, we use two models, named $\text{TransformerGen}()$ and $\text{TransformerSeg}()$, to generate the result of initial round and that of subsequent rounds. In the initial round, the input of $\text{TransformerGen}()$ is the source code sequence, and the output is the target code sequence. In the subsequent rounds, the input of $\text{TransformerSeg}()$ consists of the source code sequence as well as the user feedback. The model's output are multiple segments used to fill the validated segments. The training of $\text{TransformerGen}()$ is conducted using a program translation parallel corpus, which comprises fully aligned pairs of complete source program sequences and complete target program sequences. The training dataset for $\text{TransformerSeg}()$ consists of aligned pairs: [complete source code sequence, correct prefix, disjoint validated segments] and [corresponding target segments]. Last but not least, $\text{TransformerGen}()$ and $\text{TransformerSeg}()$ can be replaced by any transformer-based models with the required input and output as illustrated above. The training of the alternative models follows the same way.

Algorithm 4: Prefix-based Generation and Splicing method (PGS).

Input: X , the source code sequence;
 $\{y_1^t, \dots, y_{c_t}^t\}$, the confirmed prefix;
 F^t , the confirmed segments in iteration t .
Output: Y^t , the target code sequence in this iteration.

- 1 $Y = []$;
- 2 $f_{i-1} = \{y_1^t, \dots, y_{c_t}^t\}$;
- 3 f_i = the first segment in F^t ;
- 4 **while** *the last token in Y is not the terminator* **do**
- 5 $A_i^t = \text{TransformerPre}(X, Y)$;
- 6 $y_i^t = \arg \max(A_i^t)$.
- 7 **if** $f_{i-1} \neq \emptyset$ **then**
- 8 y_i^t = the next token in f_{i-1} ;
- 9 Delete this token from f_{i-1} ;
- 10 $Y = Y \oplus y_i^t$.
- 11 **else if** y_i^t equals to the first token in f_i **then**
- 12 $Y = Y \oplus y_i^t$;
- 13 $f_{i-1} = f_i[1:]$;
- 14 f_i = the next segment in F^t .
- 15 **else**
- 16 $Y = Y \oplus y_i^t$.

TABLE II
DETAILS OF THE TWO DATASETS

	Python			Java		
	# Samples	Avg	Max	# Samples	Avg	Max
AVATAR-g4g	3411	104	391	3411	109	427
GitHub	948	120	548	948	127	529

VI. EXPERIMENTS

With two real datasets, we evaluate the proposed methods from four aspects: accuracy, human effort, interaction rounds and response time. In addition, we give a user study and a case study to show the advance of our approach.

A. Experimental Settings

1) *Datasets:* We evaluate our proposed approach on two datasets:

- AVATAR dataset [36]: It includes code written in Java and Python, obtained from the open-source code contest sites and the online platforms. In the evaluation, we use a subset, that is, AVATAR-g4g that consists of Java-Python parallel functions collected from the GeeksforGeeks website.
- GitHub dataset [8]: This dataset is created and used in TransCoder [8], which consists of a set of parallel functions in C++, Java, and Python extracted from the GitHub public dataset available on Google BigQuery.

Table II shows the details of the two datasets.

2) *Metrics:* **Metrics on Accuracy:** We adopt the following metrics [37] to measure the translation accuracy.

- **Weighted N-Gram Match** ($BLEU_{weight}$): BLEU with different weights, which gives a higher weight to key tokens, e.g., keywords, but a lower weight to literals, e.g., string.
- **Syntactic AST Match** ($Match_{ast}$): measures the syntactic similarity by comparing the abstract syntax trees of the results and the target one.
- **Semantic Data-flow Match** ($Match_{df}$): measures the semantic similarity by comparing the semantic graphs of the results and the target one.

Metrics on Human Effort: We evaluate the human effort by the number of the software engineer's key strokes and mouse actions [38], [39], [40] in confirmation and correction.

- **Word Stroke Ratio (WSR)**: measures the correction action made by the human. It is defined as the ratio that the number of words corrected by the software engineer in the whole translation process, divided by the number of words in the final result Y .
- **Key Stroke Ratio (KSR)**: measures the keystroking action made by the human. It is defined as the number of the actions made by the software engineer, divided by the total number of characters in the final result Y . The number of keystroke actions is calculated as the number of characters that the software engineer corrects.
- **Mouse Action Ratio (MAR)**: measures the mouse action made by the human. This ratio is defined as the number of mouse actions made by the software engineer, divided by the total number of characters in the final result Y . Mouse action is counted by the clicking times. Correcting a token, confirming a prefix and accepting the result requires one mouse clicking. Confirming a segment requires two mouse clicking, i.e., before and after the segment.
- **Key Stroke and Mouse Action Ratio (KSMR)**: measures the keystroking and mouse clicking action made by the human. Its defined as the sum of KSR and MAR.

Metric on Interaction Rounds: The number of iterations in the whole translation process.

Metric on Response Time: Response time measures the average time cost of one iteration round between the model and the software engineer. We test the proposed methods and the baselines sequentially on one sample and then to the next one, to ensure that the computer has roughly the same loads. e.g., memory usage, volatile GPU-Util, etc.

3) **Baselines: Machine-based code translation methods:** We include three state-of-the-arts code translation models as the baselines: **TransCoder** [8], **DOBF** [9] and **CodeT5** [26].

Machine-based automated code repair methods: We include three state-of-the-arts automated code repair models as the baselines: **CoCoNut** [41], **CURE** [42] and **RewardRepair** [43]. They first make translation through a code translation model, i.e., CodeT5, and then make code repair for multiple rounds, where each round takes the output of the previous round as input, until reaches the max rounds or the code is bug-free.

Human-machine collaboration methods:

- **Human Post-Editing (HPE)**: It is a human-machine collaboration method. The method first makes machine

translation with CodeT5, and then lets the software engineer make correction.

- **Automatic Downstream Code Editing (ADCE)**: It is a UI proposed by Weisz et al. [32], which can perform relevant downstream editing when the software engineer selects an alternative translation. If the software engineer corrects the variable's name, it automatically renames all instances of the variable. Similar feature exists in some IDE tools. Similarly, we adopt CodeT5 as that used in the proposed method.
- **Interactive Translation (InterTrans)**: It is the method that is illustrated in Section III. It enables human and machine interact, but does not contain additional designed modules, i.e., Attention Cache module, Suffix Splicing module and Segment Splicing module.

4) **Settings:** First of all, we describe the target result of the translation. In some cases, the target result of the translation can be any correct code regardless of the implementation method; while in other cases, the target result of the translation is the code with specific preference. The ground truth as well as other semantically equivalent codes can be accepted as the result in the former condition [25], [30], while only the ground truth can be accepted in the latter one [8], [9]. Our evaluation fits to the latter condition since the participation of the software engineer inevitably involves his or her own programming preference, which often results in the code with specific implementation.

Then, we describe the settings of the translation system and the user simulation used in the experiment.

Translation System: CodeT5 model [26] is used as our code translation model. This model follows the same architecture as that in T5 [44], which is implemented based on the Transformers framework¹. We use the tokenizer published by CodeT5 model, which mainly follows the BPE tokenizer trained by Radford et al. [45]. The size of the vocabulary table is 32,000. CodeT5 pre-training model is fine-tuned on the AVATAR-g4g training dataset, with parameters following the settings in the source code published by Ahmad et al. [36]. In addition, we use Adam Optimizer [46] to update the model parameters with a learning rate 0.00005, a batch size 2, a hidden layer dimension 768, and a maximum training rounds 20. We finally select the model parameters with the lowest loss on the validation dataset during training process. We keep the same configuration for the training process in Python to Java and Java to Python. And in the testing part, we set the beam size to 1 and the maximum number of code tokens to 510.

User Simulation: In the evaluation, we introduce user simulator following the automated simulation scheme [15], [38] due to the high cost and low efficiency. In addition, experiments with real software engineers are also included as a supplement. In the simulation, we assume that the target code sequence corresponding to the source code in the parallel corpus is used as the final result that the software developer wants to obtain. (1) In the interactive code translation scenario, since it is a prefix-based interactive protocol, we compare the current translation result with the target code sequence token by token and find

¹<https://huggingface.co/docs/transformers/index>

TABLE III
COMPARISON WITH THE MACHINE-BASED METHODS ON METRICS OF ACCURACY (AVATAR-G4G DATASET)

Direction Metric	Python to Java			Java to Python		
	BLEU _{weight}	Match _{ast}	Match _{df}	BLEU _{weight}	Match _{ast}	Match _{df}
TransCoder	74.73	72.8	71.15	72.64	53.3	59.9
TransCoder-DOBF	74.81	72.7	71.99	73.39	53.71	57.75
CodeT5	82.1	80.14	76.16	82.21	62.32	58.3
CoCoNuT	83.15	86.29	77.14	88.01	85.52	74.15
CURE	86.65	88.95	78.83	91.59	89.69	80.01
RewardRepair	88.09	90.42	79.83	92.88	91.45	82.25
Prefix-based hmCodeTrans	100	100	100	100	100	100
Segment-based hmCodeTrans	100	100	100	100	100	100

TABLE IV
COMPARISON WITH THE MACHINE-BASED METHODS ON METRICS OF ACCURACY (GITHUB DATASET)

Direction Metric	Python to Java			Java to Python		
	BLEU _{weight}	Match _{ast}	Match _{df}	BLEU _{weight}	Match _{ast}	Match _{df}
TransCoder	74.56	72.76	69.17	71.71	55.6	64.44
TransCoder-DOBF	74.32	72.44	68.95	74.07	58.22	63.62
CodeT5	81.66	79.52	71.22	82.62	67.69	67.56
CoCoNuT	82.54	85.37	73.87	88.49	87.03	76.45
CURE	86.34	88.48	77.03	92.40	91.55	81.84
RewardRepair	87.78	90.00	78.50	93.44	92.65	84.26
Prefix-based hmCodeTrans	100	100	100	100	100	100
Segment-based hmCodeTrans	100	100	100	100	100	100

out the first token that does not match. Then, we assume the software engineer makes correction at this position and the corrected token is obtained from the target code. An exact match between the translation result and the target will be considered as the acceptance from the software engineer. (2) For the HPE method, we use the translation result of the first round as the output that the software engineer needs to edit. (3) For the ADCE method, the best way to simulate is on a coding IDE, which, however, is difficult. Consequently, we consider an approximation. Since it is not convenient to distinguish the variables directly, we consider distinguishing a superset of variables, i.e., identifiers. We adopt the syntax parsing tool called tree-sitter² to parse the code and obtain a list of the identifiers to be replaced. The difference between the number of identifiers in the original list and the number of identifiers in the de-duplicated list is approximately the same to the software engineer's reduced edit action.

B. Results and Analysis

1) *Results and Analysis on Accuracy:* Table III and Table IV provide the result of comparison with the machine-based code translation methods and the machine-based automated code repair methods on the metrics of accuracy, i.e., BLEU_{weight}, Match_{ast} and Match_{df}, in AVATAR-g4g dataset and GitHub dataset, respectively. From the results we can observe that

- The automated code repair methods perform better than the code translation methods. This is because the latter ones conduct code repair based on the initial translation

results given by the former ones. However, even the best machine-based methods can only achieve a less than 93% accuracy, which can not be directly used in real-world applications.

- Compared to these purely machine-based baselines, the proposed human-machine collaboration methods show superiority in code translation accuracy. Specifically, it can guarantee an error-free (100% accuracy) result.

2) *Results and Analysis on Human Effort:* The results on the metrics WSR, KSR, MAR as well as KSMR are provided in Table V, Table VI, Table VII and Table VIII respectively. From the results we can observe that

- Compared with HPE and ADCE, hmCodeTrans can reduce great human effort in terms of WSR and KSR. As for MAR, prefix-based hmCodeTrans has a better performance compared with HPE and ACDE; segment-based hmCodeTrans requires more key stroke actions. This is because that they require confirmation from the software engineer, which is completed through mouse clicking. Finally, their performance in KSMR shows that hmCodeTrans can reduce human effort with a combination of key stroke and mouse click.
- Compared with the interactive method InterTrans, prefix-based hmCodeTrans owns an approximately similar performance, which indicates that the abort point searching algorithm in suffix splicing module can always contain at least one wrong token and therefore has little effect on its performance. Segment-based hmCodeTrans requires less key stroke actions to make correction, while more mouse clicking actions to make confirmation.

²<https://tree-sitter.github.io/tree-sitter/>

TABLE V
RESULTS ON THE METRIC WORD STROKE RATIO (WSR)

Direction Code Length	AVATAR-g4g						GitHub					
	Python to Java			Java to Python			Python to Java			Java to Python		
	< 100	100-200	> 200	< 100	100-200	> 200	< 100	100-200	> 200	< 100	100-200	> 200
HPE	0.129	0.156	0.222	0.102	0.133	0.181	0.149	0.162	0.195	0.102	0.133	0.181
ADCE	0.128	0.152	0.213	0.099	0.13	0.173	0.146	0.157	0.188	0.099	0.13	0.173
InterTrans	0.038	0.043	0.054	0.033	0.034	0.044	0.045	0.040	0.047	0.04	0.037	0.041
Prefix-based hmCodeTrans	0.038	0.044	0.055	0.033	0.035	0.045	0.045	0.041	0.048	0.04	0.038	0.041
Segment-based hmCodeTrans	0.028	0.026	0.03	0.022	0.02	0.028	0.032	0.024	0.028	0.028	0.022	0.023

TABLE VI
RESULTS ON THE METRIC KEY STROKE RATIO (KSR)

Direction Code Length	AVATAR-g4g						GitHub					
	Python to Java			Java to Python			Python to Java			Python to Java		
	< 100	100-200	> 200	< 100	100-200	> 200	< 100	100-200	> 200	< 100	100-200	> 200
HPE	0.128	0.157	0.226	0.088	0.116	0.164	0.15	0.167	0.198	0.088	0.116	0.164
ADCE	0.126	0.15	0.212	0.084	0.112	0.154	0.145	0.158	0.187	0.084	0.112	0.154
InterTrans	0.048	0.053	0.069	0.032	0.034	0.047	0.054	0.050	0.060	0.044	0.039	0.042
Prefix-based hmCodeTrans	0.048	0.055	0.070	0.032	0.034	0.048	0.055	0.051	0.061	0.045	0.04	0.043
Segment-based hmCodeTrans	0.035	0.034	0.041	0.022	0.02	0.03	0.04	0.031	0.038	0.031	0.023	0.024

TABLE VII
RESULTS ON THE METRIC MOUSE ACTION RATIO (MAR)

Direction Code Length	AVATAR-g4g						GitHub					
	Python to Java			Java to Python			Python to Java			Python to Java		
	< 100	100-200	> 200	< 100	100-200	> 200	< 100	100-200	> 200	< 100	100-200	> 200
HPE	0.053	0.061	0.086	0.041	0.048	0.066	0.06	0.063	0.078	0.041	0.048	0.066
ADCE	0.053	0.059	0.083	0.04	0.047	0.063	0.059	0.062	0.075	0.04	0.047	0.063
InterTrans	0.034	0.035	0.043	0.029	0.026	0.033	0.038	0.033	0.038	0.032	0.028	0.031
Prefix-based hmCodeTrans	0.034	0.035	0.043	0.029	0.026	0.034	0.039	0.033	0.039	0.033	0.029	0.031
Segment-based hmCodeTrans	0.056	0.092	0.204	0.04	0.049	0.164	0.071	0.084	0.183	0.051	0.058	0.129

TABLE VIII
RESULTS ON THE METRIC KEY STROKE AND MOUSE ACTION RATIO (KSMR)

Direction Code Length	AVATAR-g4g						GitHub					
	Python to Java			Java to Python			Python to Java			Python to Java		
	< 100	100-200	> 200	< 100	100-200	> 200	< 100	100-200	> 200	< 100	100-200	> 200
HPE	0.181	0.218	0.312	0.129	0.165	0.23	0.21	0.23	0.276	0.129	0.165	0.23
ADCE	0.179	0.209	0.294	0.125	0.159	0.217	0.204	0.22	0.262	0.125	0.159	0.217
InterTrans	0.082	0.088	0.111	0.061	0.059	0.08	0.093	0.083	0.098	0.077	0.067	0.073
Prefix-based hmCodeTrans	0.083	0.09	0.113	0.061	0.061	0.081	0.094	0.085	0.1	0.077	0.068	0.074
Segment-based hmCodeTrans	0.091	0.126	0.245	0.063	0.069	0.194	0.112	0.116	0.22	0.082	0.081	0.154

- With the increase of code length, the outperformance of hmCodeTrans becomes greater. For example, for the codes whose length is less than 100, lies between 100 to 200 and is greater than 200, compared with HPE, prefix-based hmCodeTrans owns a KSMR reduction of 54.1%, 58.7% and 63.8% respectively, which shows its potential advance in long code translation.
- Prefix-based hmCodeTrans and segment-based hmCodeTrans have the significant difference. Combine the results in Table VI and Table VII, we can observe that

TABLE IX
RESULTS ON THE METRIC INTERACTION ROUNDS

Direction Code Length	AVATAR-g4g						GitHub					
	Python to Java			Java to Python			Python to Java			Python to Java		
	< 100	100-200	> 200	< 100	100-200	> 200	< 100	100-200	> 200	< 100	100-200	> 200
CoCoNuT	3.487	9.062	16.942	3.115	7.808	15.626	3.580	8.123	17.371	3.588	6.607	15.471
CURE	2.769	7.506	15.760	2.715	5.636	13.968	3.429	7.322	15.98	3.294	6.339	13.494
RewardRepair	2.569	6.888	15.093	2.268	5.108	13.28	2.956	6.636	15.45	3.176	5.717	12.473
InterTrans	2.382	6.089	14.162	2.192	4.839	12	2.84	5.81	12.7	2.636	5.386	11.377
Prefix-based hmCodeTrans	2.402	6.215	14.324	2.211	4.975	12.301	2.880	5.920	12.87	2.649	5.498	11.551
Segment-based hmCodeTrans	1.696	3.746	7.924	1.451	2.882	7.804	1.97	3.49	7.64	1.78	3.219	6.675

TABLE X
STANDARD DEVIATION OF INTERACTION ROUNDS OF CODES
WITH LENGTH [90, 110]

Direction	AVATAR-g4g		GitHub	
	P2J	J2P	P2J	P2J
Prefix-based hmCodeTrans	3.458	3.216	3.278	3.916
Segment-based hmCodeTrans	2.159	1.736	2.191	2.502

prefix-based hmCodeTrans requires less key stroke actions (correction actions) while segment-based hmCodeTrans requires less mouse clicking actions (confirmation actions). We assume these two actions consume equal human effort, and thus the metric KSMR demonstrates a total human effort exertion. From the results in Table VIII we can conclude that segment-based hmCodeTrans requires more human effort compared with prefix-based hmCodeTrans.

3) *Results and Analysis on Interaction Rounds:* For the automated code repair methods, we set the max rounds of them to 10, 15 and 20 for the code whose length is less than 100, lies between 100 to 200 and is greater than 200, respectively. The results on the metric interaction rounds are provided in Table IX. From the results we can observe that

- The proposed hmCodeTrans has a smaller number of interaction rounds compared with the automated code repair methods. In addition, Prefix-based hmCodeTrans has an approximately similar performance compared with InterTrans, which indicates our suffix splicing module has little effect on the results. Segment-based hmCodeTrans has a lower interaction rounds compared with InterTrans.
- For the two proposed methods, segment-based hmCodeTrans performs better than prefix-based hmCodeTrans in terms of the metric interaction rounds, which, however, involves more human effort as illustrated before.
- To evaluate the stability, we calculate the standard deviation for the interaction rounds of codes with length from 90 to 110. The results are given in Table X. From the results we can conclude that the stability of the proposed methods is acceptable but not very good. This is resulted by the instability of data sample (varying number of errors in each sample) and human feedback (varying correction strategy).

4) *Results and Analysis on Response Time:* The results on the metric response time are demonstrated in Table XI. From the results we can observe that

- Compared with InterTrans, hmCodeTrans has a significant reduction on response time.
- For prefix-based hmCodeTrans, the contribution of attention cache module and suffix splicing module in terms of response time is given in Table XII. Attention cache module and suffix splicing module can greatly help to reduce the response time.
- Fig. 5 illustrates the response time of the codes with varying code length. From the figure we can observe that InterTrans has largest response time. Prefix-based hmCodeTrans and segment-based hmCodeTrans have a small response time. What's more, the gap between InterTrans and our proposed methods becomes larger when code length increases, which shows the proposed methods' advance in translation of long codes.
- Fig. 6 and Fig. 7 show response time in each round during the translation for the codes with total rounds 5 and 10 respectively. From the figures we can observe that prefix-based hmCodeTrans and segment-based hmCodeTrans have lower response time in each round compared with InterTrans. In addition, we can observe the effect of attention cache module and suffix splicing module from the figure - attention cache helps to reduce response time in later rounds since its usage is to skip the confirmed prefix that becomes longer with the increase of interaction rounds; suffix splicing helps to reduce response time in early rounds since its usage is to skip the useless inference of suffix.

5) *Some Additional Results and Analysis: Results on prefix-based hmCodeTrans.* We first evaluate whether the abort point selected by suffix splicing module is an appropriate one and is splicable. Specifically, we discuss (i) whether the inferred tokens before the abort point have included at least one wrong token and (ii) is this abort point the best to splice among all the possible splicing positions.

- We evaluate the ratio of the rounds in which the inferred tokens before the abort point have included at least one wrong token to all the rounds on the two datasets in both Python to Java and Java to Python directions. The average

TABLE XI
RESULTS ON THE METRIC RESPONSE TIME

Direction Code Length	AVATAR-g4g						GitHub					
	Python to Java			Java to Python			Python to Java			Python to Java		
	< 100	100-200	> 200	< 100	100-200	> 200	< 100	100-200	> 200	< 100	100-200	> 200
InterTrans	2.402	4.858	8.934	2.748	5.722	11.149	2.407	4.961	9.37	2.708	5.453	11.08
Prefix-based hmCodeTrans	0.86	1.524	2.43	1.015	1.902	3.46	1.025	1.67	2.669	1.041	1.822	3.167
Segment-based hmCodeTrans	0.202	0.444	1.444	0.368	0.623	3.183	0.244	0.41	1.438	0.507	1.322	1.932

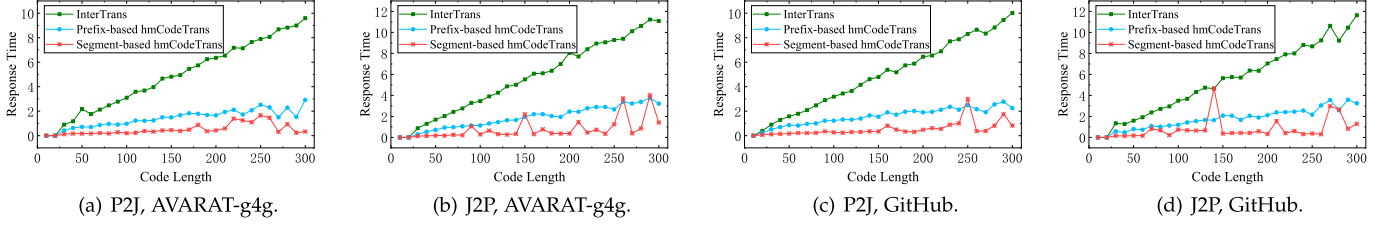


Fig. 5. Response time of the codes with varying code length.

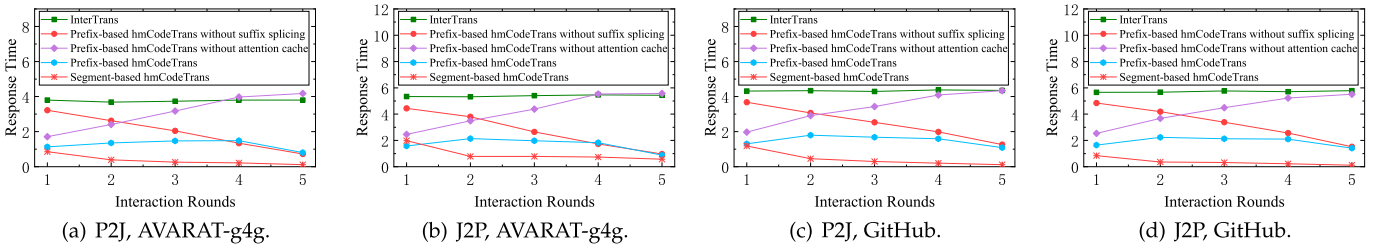


Fig. 6. Response time in each round of the codes with total rounds 5.

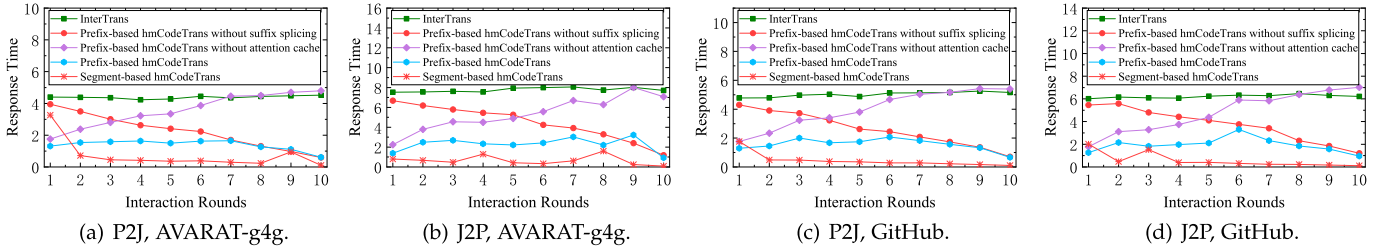


Fig. 7. Response time in each round of the codes with total rounds 10.

TABLE XII
EVALUATION ON THE EFFECT OF ATTENTION CACHE MODULE
AND SUFFIX SPLICING MODULE IN TERMS OF RESPONSE TIME,
WHERE THE CODE LENGTH LIES BETWEEN 100 AND 200

Direction	AVATAR-g4g		GitHub	
	P2J	J2P	P2J	J2P
Prefix-based hmCodeTrans without attention cache	3.884	4.564	3.992	4.36
Prefix-based hmCodeTrans without suffix splicing	2.603	2.973	2.474	2.894
Prefix-based hmCodeTrans	1.524	1.902	1.670	1.822

value of the ratio is 92.1%, which indicates that in most cases the selected abort position is an appropriate one.

- We evaluate the ratio of the rounds in which the selected abort point is the best one to splice among all the possible splicing positions to all the rounds. In specific, a splicing position is a possible one if it satisfies condition 1) as in Equation (7), and it is the best one if it has the highest cosine similarity, as given in Equation (8), among all the possible ones. The average value of the ratio is 94.7%, which indicates that the proposed algorithm can find the best position to splice in most cases.

TABLE XIII
KMSR AND RESPONSE TIME WITH DIFFERENT CLASSIFIERS AND INCREMENTS, WHERE THE CODE LENGTH LIES BETWEEN 100 AND 200

	KMSR				Response Time			
	AVATAR-g4g		GitHub		AVATAR-g4g		GitHub	
	P2J	J2P	P2J	J2P	P2J	J2P	P2J	J2P
Prefix-based hmCodeTrans - SVM - 20	0.091	0.062	0.087	0.069	1.313	3.254	1.454	2.986
Prefix-based hmCodeTrans - GBDT - 20	0.09	0.061	0.085	0.068	1.511	3.503	1.649	3.242
Prefix-based hmCodeTrans - AdaBoost - 10	0.091	0.061	0.085	0.069	1.35	3.125	1.5	2.899
Prefix-based hmCodeTrans - AdaBoost - 20	0.09	0.061	0.085	0.068	1.524	3.46	1.67	3.167
Prefix-based hmCodeTrans - AdaBoost - 30	0.089	0.061	0.084	0.068	1.682	3.73	1.81	3.404
Prefix-based hmCodeTrans - AdaBoost - 40	0.089	0.06	0.084	0.068	1.784	3.924	1.931	3.642
Prefix-based hmCodeTrans - AdaBoost - 50	0.089	0.06	0.084	0.068	1.897	4.168	2.05	3.863

Note: Bold indicates the best performance among all the experimental results.

TABLE XIV
HUMAN EFFORT (KSR, MAR AND KSMR) AND RESPONSE TIME OF THE TWO CASES IN SEGMENT-BASED hmCodeTRANS

	KSR				MAR				KSMR				Response Time			
	AVATAR-g4g		GitHub		AVATAR-g4g		GitHub		AVATAR-g4g		GitHub		AVATAR-g4g		GitHub	
	P2J	J2P	P2J	J2P	P2J	J2P	P2J	J2P	P2J	J2P	P2J	J2P	P2J	J2P	P2J	J2P
Case 1	0.029	0.02	0.028	0.021	0.073	0.057	0.073	0.056	0.102	0.077	0.101	0.077	0.336	1.294	0.345	0.882
Case 2	0.018	0.014	0.018	0.011	0.246	0.248	0.245	0.224	0.264	0.261	0.262	0.235	10.223	12.631	10.706	14.347

Subsequently, we evaluate how the type of classifier and the increment setting affect the results. The results are given in Table XIII, in which the methods are named as “Segment-based hmCodeTrans - classifier - increment”.

- To evaluate how the type of classifier affects the results, we compare rows 1, 2, and 4 in Table XIII. For human effort, due to the limited space, we evaluate the methods with different classifiers on codes with length from 100 to 200 and take the metric KMSR into consideration. From the results we can find methods that use SVM, GBDT and AdaBoost have roughly the same performance. For response time, from the results we can observe that GBDT takes a little more time, while SVM and GBDT have roughly the same response time.
- To evaluate how the increment setting affects the results, we compare rows 3 to 7 in Table XIII. Considering human effort, KMSR decreases with the increase of increment, but does not decrease any more when the increment is larger than 30. This is because if the increment is large enough, the inferred tokens must include at least one wrong token. For response time, it increases with the increase of increment. This is because a larger increment indicates a longer inference, and thus takes more time.

Results on segment-based hmCodeTrans. For segment-based hmCodeTrans, we evaluate the occurrence frequency of case 1, i.e., the number of generated segments m_i^t equals to that of confirmed segments f_i^{t-1} , in which we can splice the two parts of segments directly, and case 2 in which we make splicing with the proposed heuristic splicing method. In addition, we conduct experiments to show how segment-based hmCodeTrans performs in terms of human effort and response time in the two cases.

- We calculate the occurrence frequency of case 1 on the two datasets with two translation directions. The average value

of the frequency is 95.9%, which indicates the translation results falling into case 1 occurs in most cases. This promotes the advance of segment-based hmCodeTrans since case 1 is more effective.

- Human effort and response time in the two cases are given in Table XIV. For human effort, we can observe that case 2 takes a certain decrease on KSR but a significant increase on MAR, and eventually results in a high KSMR compared with that in case 1. For response time, case 2 also takes a huge response time with over 10s. However, thanks to the low occurrence ratio of case 2, i.e., about 4.1%, it has little impact on the whole performance of segment-based hmCodeTrans.

C. Experiments With Human Interaction

To explore the real human-machine collaboration in our proposed framework, we invite five software engineers to conduct real interaction experiments.

1) *Baselines*: The proposed prefix-based and segment-based patterns require the software engineer to make correction on just one token in a sequential manner, which may not well meet the software engineer’s habit. In order to show why we make such settings and what will happen if we loosen the above requirements, we introduce two other feedback patterns:

- **hmCodeTrans without quantity restriction**: In this pattern, there exists no limit on the number of tokens that a user can correct. Users can correct sub-word components or erroneous expressions in each iteration.
- **hmCodeTrans without location restriction**: In this pattern, there exists no limit on the location of the token that a user can correct. Users can correct the wrong token in any location, not necessarily the first one.

TABLE XV
RESULTS OF THE REAL PREFIX-BASED INTERACTION SCENARIO

Metrics	Interaction Rounds	WSR	KMSR	Collaboration Time
Prefix-based hmCodeTrans	2.6	0.022	0.068	24.396
Prefix-based hmCodeTrans without quantity restriction	2.3	0.039	0.133	19.896
Prefix-based hmCodeTrans without location restriction	3.04	0.028	0.12	18.199

Note: Bold indicates the best performance among all the experimental results.

TABLE XVI
RESULTS OF THE REAL SEGMENT-BASED INTERACTION SCENARIO

Metrics	Interaction Rounds	WSR	KMSR	Collaboration Time
Segment-based hmCodeTrans	2.12	0.018	0.096	25.112
Segment-based hmCodeTrans without quantity restriction	2.08	0.032	0.159	21.299
Segment-based hmCodeTrans without location restriction	2.2	0.022	0.129	19.74

Note: Bold indicates the best performance among all the experimental results.

The implementation of these two variants are the same as the standard one. After receiving feedback, the model first gives a constraint decoding for the tokens that have been confirmed/corrected by the developer, then proceeds a free decoding for the remaining tokens. The difference lies in the range - compared with the standard method, the number and the position of the corrected tokens will change in hmCodeTrans without quantity restriction and that without location restriction respectively, which therefore results in a different constraint decoding range.

2) *Settings*: We invited five real software engineers as the participants. The five software engineers have at least four years of programming experience and are familiar with Java and Python. In our experiment, we ask these five participants to perform human-machine code translation following the standard pattern as well as the two variant patterns. Specifically, we exacted 60 samples, whose lengths are all around 100 in order to ensure a same difficulty, from the test set of AVATAR-g4g dataset. Then, randomly separated them into six groups for the 6 sub-tasks as listed in Table XV and Table XVI. The tasks for all the participants are the same.

On one hand, we make objective evaluation with the metrics as given before. Note we replace response time (the time of machine retranslation) with collaboration time (the sum of the time of machine retranslation and human feedback). On the other hand, we make subjective evaluation by releasing a questionnaire to collect their user experience. Our questionnaire contains the following four questions:

- Question 1: Do you think the required actions are difficult or time-consuming in the prefix-based and segment-based hmCodeTrans patterns?
- Question 2: Do you think the ease of action has been improved by removing quantity restriction?
- Question 3: Do you think the ease of action has been improved by removing location restriction?
- Question 4: Are you willing to increase labor to gain more action freedom?

3) *Results and Analysis*: The results of the objective evaluation are shown in Table XV and Table XVI. From the results, we can observe that the proposed collaboration patterns own

the lowest human effort in terms of WSR and KMSR. To be more specific

- hmCodeTrans without quantity restriction increases the overall efficiency but involves more human effort. The reason lies in that the software engineers make more corrections, which introduces additional burdens to human but provides more information to model. In the standard hmCodeTrans, the model automatically corrects the next token based on the one given by the user.
- hmCodeTrans without location restriction decreases the collaboration time but increases human effort and interaction rounds. This is because the location of corrected token affects the retranslation of its suffix tokens. For example, for a program with 100 tokens, a correction at position 10 can help to update the remaining 90 tokens behind it but a correction at position 80 can only help to update the remaining 20 tokens behind it. Therefore, hmCodeTrans without location restriction is friendly to human but unfriendly to model.

The results of the subjective evaluation are shown in Fig. 8.

- For question 1, from the results we can see that the prefix-based and segment-based human-machine cooperation methods we proposed are user-friendly. The whole process is not too difficult or time-consuming for the user.
- For question 2 and 3, from the results we can see that the users own more ease of action. But from the results in Table XV and Table XVI we can see that the ease is followed by the increase of human effort.
- For question 4, it can be seen from the results that different users have different preferences in action freedom and labor saving.

In conclusion, the standard hmCodeTrans helps to save human effort while the two relaxed versions are more user friendly.

D. Case Study

We give two typical examples for prefix-based hmCodeTrans and segment-based hmCodeTrans respectively.

1) *Example for Prefix-Based hmCodeTrans*: We present the interaction process of No. 609 sample in AVATAR-g4g dataset for example.

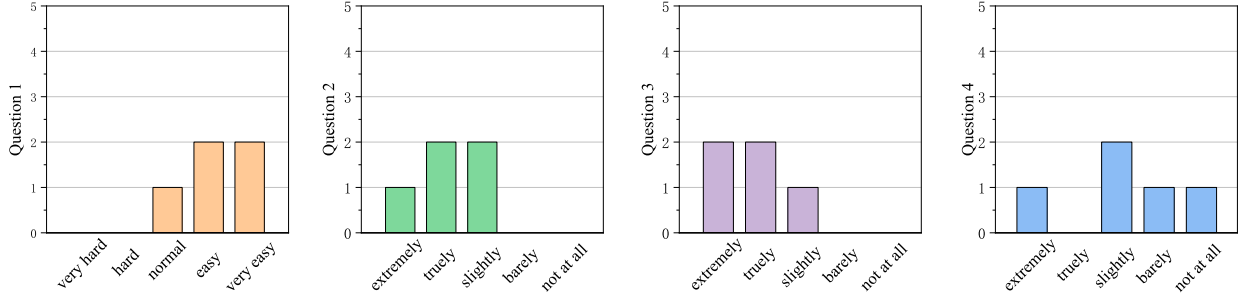


Fig. 8. The results of the questionnaire.

Initial code translation made by machine

	Code in source language	Initial result in target language
Round 0	<pre>def checksum (n) : n = int (n) if n % 3 == 0 : print (int (n / 3 - 1) , " _ " , int (n / 3) , " _ " , int (n / 3 + 1)) else : print (" -1 ")</pre>	<pre>static void checksum (int n) { n = n % 3 ; if (n % 3 == 0) System.out.println ((n / 3 - 1) + " _ " + (n / 3) + " _ ") ; else System.out.println (" -1 ") ; }</pre>
		2.52s

Human

Machine

Round 1	<pre>static void checksum (int n) { if n % 3 ; if (n % 3 == 0) System.out.println ((n / 3 - 1) + " _ " + (n / 3) + " _ ") ; else System.out.println (" -1 ") ; }</pre>	Human feedback in round 1	<pre>static void checksum (int n) { if (n % 3 == 0) System.out.println ((n / 3 - 1) + " _ " + (n / 3) + " _ ") ; else System.out.println (" -1 ") ; }</pre>	Model regeneration in round 1	0.67s
Round 2	<pre>static void checksum (int n) { if (n % 3 == 0) System.out.println ((n / 3 - 1) + " _ " + (n / 3) + " _ ") ; else System.out.println (" -1 ") ; }</pre>	Human feedback in round 2	<pre>static void checksum (int n) { if (n % 3 == 0) System.out.println ((n / 3 - 1) + " _ " + (n / 3) + " _ ") ; else System.out.println (" -1 ") ; }</pre>	Model regeneration in round 2	1.22s
Round 3	<pre>static void checksum (int n) { if (n % 3 == 0) System.out.println ((n / 3 - 1) + " _ " + (n / 3) + " _ " + (n / 3 + 1)) ; else System.out.println (" -1 ") ; }</pre>	Human feedback in round 3	<pre>static void checksum (int n) { if (n % 3 == 0) System.out.println ((n / 3 - 1) + " _ " + (n / 3) + " _ " + (n / 3 + 1)) ; else System.out.println (" -1 ") ; }</pre>	Model regeneration in round 3	0.71s

Fig. 9. Case study on prefix-based hmCodeTrans. In round 0, the model makes initial translation. Then, in each round, human makes confirmation on the correct tokens (colored with green) and correction on the wrong token (colored with red), based on which machine updates the next few tokens (not colored) and splices the predefined suffix (colored with blue). Repeat until all the tokens are accepted.

As shown in Fig. 9, the initial translation result generated by the model has 14 wrong tokens, including 6 redundant ones, i.e., $n = n \% 3$; and 8 missing ones, i.e., $+(n / 3 + 1)$. In the first round, the software engineer checks from the beginning, labels the right tokens (colored with green) and corrects the first wrong token from “n” to “if” (colored with red), based on which the model retranslates and deletes the remaining redundant tokens automatically. The model continues retranslating until the token “_” - the position it believes the inferred tokens have included at least one wrong token and the current position is splicable. And finally, the model splices the inferred tokens (not colored) with the predefined suffix that is generated in the initial translation (colored with blue). Repeat until all the tokens are accepted. In this example, among the 14 wrong tokens, 3 are corrected by the software engineer and 11 are corrected by the model automatically based on the feedback from human, which helps to reduce 78.6% human effort.

2) *Example for Segment-Based hmCodeTrans:* We illustrate the interaction process of No. 253 sample in GitHub dataset for example.

As shown in Fig. 10, the model gives an initial translation result that includes 8 wrong tokens, i.e., 1 incorrect one, i.e., return should be if, and 7 missing ones, i.e., public and return false ; else return ture. In round 1, the software engineer starts to check and correct the first wrong token he/she meets, i.e., from “static” to “public” (colored with red), which is similar as in prefix-based hmCodeTrans. What’s different, the software engineer does not stop after correction but keeps checking and labels the correct tokens in the remaining part, which form 3 segments (colored with green). Based on the confirmed prefix and segments given by the software engineer, then the model generates the token fragments to fill between the confirmed segments (not colored). Repeat until all the tokens are accepted. In this example, among the 8

Initial code translation made by machine		
	Code in source language	Initial result in target language
Round 0	<pre>def isEven (int n) : return (not (n & 1)) }</pre>	<pre>static boolean isEven (int n) { return ((n & 1) != 0) ; }</pre> 1.21s
Human Machine		
Round 1	<pre>public boolean isEven (int n) { return ((n & 1) != 0) ; }</pre>	<pre>public static boolean isEven (int n) { return ((n & 1) != 0) ; }</pre> 0.19s
Round 2	<pre>public static boolean isEven (int n) { if ((n & 1) != 0) ; }</pre>	<pre>public static boolean isEven (int n) { if ((n & 1) != 0) return false ; else return true ; }</pre> 0.25s

Fig. 10. Case study on segment-based hmCodeTrans. In round 0, the model makes initial translation. Then, in each round, human makes confirmation on the correct tokens (colored with green) and correction on the wrong token (colored with red), based on which machine updates the next few tokens (not colored). Repeat until all the tokens are accepted.

wrong tokens, 2 are corrected by the software engineer and 6 are corrected by the model, which reduces 75.0% human effort.

VII. CONCLUSION

This work proposes hmCodeTrans, a novel method that achieves code translation in an interactive human-machine collaboration way. Specifically, on one hand, we introduce two human-machine collaboration patterns: prefix-based and segment-based ones, which can greatly save human effort compared with post-editing translation; on the other hand, we design two modules, i.e., attention cache module and suffix splicing module, to avoid duplicate inference of confirmed prefix and invalid suffix respectively, which can greatly reduce response time compared with interactive translation without the proposed modules. Experiments are conducted on real datasets. Results show compared to the baselines, our approach saves the human effort and reduces the response time effectively.

REFERENCES

- [1] C. Sharma, "FLUX: From SQL to GQL query translation tool," in *Proc. 35th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, 2020, pp. 1379–1381.
- [2] B. Chen and Z. Abedjan, "Interactive cross-language code retrieval with auto-encoders," in *Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, 2021, pp. 167–178.
- [3] J. Chi, Y. Qu, T. Liu, Q. Zheng, and H. Yin, "SeqTrans: Automatic vulnerability fix via sequence to sequence learning," *IEEE Trans. Softw. Eng.*, vol. 49, no. 2, pp. 564–585, Feb. 2023.
- [4] K. Alsubhi et al., "A tool for translating sequential source code to parallel code written in C++ and OpenACC," in *Proc. IEEE/ACS 16th Int. Conf. Comput. Syst. Appl. (AICCSA)*, 2019, pp. 1–8.
- [5] K. An, N. Meng, and E. Tilevich, "Automatic inference of Java-to-Swift translation rules for porting mobile applications," in *Proc. 5th Int. Conf. Mobile Softw. Eng. Syst. (MobileSoft)*, 2018, pp. 180–190.
- [6] S. Pino, L. Pollock, and S. Chandrasekaran, "Exploring translation of OpenMP to OpenACC 2.5: Lessons learned," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, 2017, pp. 673–682.
- [7] M. Tauda, Z. Zainuddin, and Z. Tahir, "Programming language translator for integration client application with web APIs," in *Proc. Int. Conf. Artif. Intell. Mechatron. Syst. (AIMS)*, 2021, pp. 1–4.
- [8] B. Roziere, M.-A. Lachaux, L. Chausson, and G. Lample, "Unsupervised translation of programming languages," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 20601–20611.
- [9] M.-A. Lachaux, B. Roziere, M. Szafraniec, and G. Lample, "DOBF: A deobfuscation pre-training objective for programming languages," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 2021, vol. 34, pp. 14967–14979.
- [10] X. Chen, C. Liu, and D. Song, "Tree-to-tree neural networks for program translation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, pp. 2547–2557, 2018.
- [11] Z. Yu, Q. Li, F. Yang, and B. Guo, "Human-machine computing," *CCF Trans. Pervasive Comput. Interact.*, vol. 3, no. 1, pp. 1–12, 2021.
- [12] C. Firestone, "Performance vs. competence in human-machine comparisons," *Proc. Nat. Acad. Sci. (PNAS)*, vol. 117, no. 43, pp. 26562–26571, 2020.
- [13] S. Seidel, N. Berente, A. Lindberg, K. Lyytinen, and J. V. Nickerson, "Autonomous tools and design: A triple-loop approach to human-machine learning," *Commun. ACM*, vol. 62, no. 1, pp. 50–57, 2018.
- [14] Á. Peris, M. Domingo, and F. Casacuberta, "Interactive neural machine translation," *Comput. Speech Lang.*, vol. 45, pp. 201–220, 2017.
- [15] A. Peris and F. Casacuberta, "Online learning for effort reduction in interactive neural machine translation," *Comput. Speech Lang.*, vol. 58, pp. 98–126, 2019.
- [16] K. K. Gupta, R. Haque, A. Ekbal, P. Bhattacharyya, and A. Way, "Syntax-informed interactive neural machine translation," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2020, pp. 1–8.
- [17] J. Heer, "Agency plus automation: Designing artificial intelligence into interactive systems," *Proc. Nat. Acad. Sci. (PNAS)*, vol. 116, no. 6, pp. 1844–1850, 2019.
- [18] A. Mastropaolo et al., "Using transfer learning for code-related tasks," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 1580–1598, Apr. 2023.
- [19] Q. Wang et al., "Learning deep transformer models for machine translation," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2019, pp. 1810–1822.
- [20] Z. Zhao et al., "ProTo: Program-guided transformer for program-guided tasks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 17021–17036.
- [21] S. Karaivanov, V. Raychev, and M. Vechev, "Phrase-based statistical translation of programming languages," in *Proc. ACM Int. Symp. New Ideas, New Paradigms, Reflections Program. Softw. (SPLASH)*, 2014, pp. 173–184.
- [22] A. T. Nguyen, T. T. Nguyen, and T. N. Nguyen, "Lexical statistical machine translation for language migration," in *Proc. 9th Joint Meeting Found. Softw. Eng. (ESEC/FSE)*, 2013, pp. 651–654.
- [23] A. T. Nguyen, T. T. Nguyen, and T. N. Nguyen, "Divide-and-conquer approach for multi-phase statistical migration for source code (T)," in *Proc. 30th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, 2015, pp. 585–596.
- [24] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," 2022, *arXiv:2204.02311*.
- [25] M. Szafraniec, B. Roziere, H. L. F. Charton, P. Labatut, and G. Synnaeve, "Code translation with compiler representations," 2022, *arXiv:2207.03578*.

- [26] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," 2021, *arXiv:2109.00859*.
- [27] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," 2020, *arXiv:2002.08155*.
- [28] X. Zhou, D. Han, and D. Lo, "Assessing generalizability of CodeBERT," in *Proc. IEEE Int. Conf. Softw. Maintenance Evolution (ICSME)*, 2021, pp. 425–436.
- [29] A. Karmakar and R. Robbes, "What do pre-trained code models know about code?" in *Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, 2021, pp. 1332–1336.
- [30] B. Roziere, J. M. Zhang, F. Charton, M. Harman, G. Synnaeve, and G. Lample, "Leveraging automated unit tests for unsupervised code translation," 2021, *arXiv:2110.06773*.
- [31] J. D. Weisz et al., "Better together? An evaluation of AI-supported code translation," in *Proc. 27th ACM Int. Conf. Intell. User Interfaces (IUI)*, 2022, pp. 369–391.
- [32] J. D. Weisz et al., "Perfection not required? Human-AI partnerships in code translation," in *Proc. 26th Int. Conf. Intell. User Interfaces (IUI)*, 2021, pp. 402–412.
- [33] G. Foster, P. Isabelle, and P. Plamondon, "Target-text mediated interactive machine translation," *Mach. Transl.*, vol. 12, no. 1, pp. 175–194, 1997.
- [34] T. Zhao et al., "Balancing quality and human involvement: An effective approach to interactive neural machine translation," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2020, vol. 34, no. 5, pp. 9660–9667.
- [35] Á. Peris and F. Casacuberta, "Active learning for interactive neural machine translation of data streams," in *Proc. 22nd Conf. Comput. Natural Lang. Learn. (CoNLL)*, 2018, pp. 151–160.
- [36] W. U. Ahmad, M. G. R. Tushar, S. Chakraborty, and K.-W. Chang, "AVATAR: A parallel corpus for Java-Python program translation," 2021, *arXiv:2108.11590*.
- [37] S. Ren et al., "CodeBLEU: A method for automatic evaluation of code synthesis," 2020, *arXiv:2009.10297*.
- [38] J. Tomás and F. Casacuberta, "Statistical phrase-based models for interactive computer-assisted translation," in *Proc. COLING/ACL Main Conf. Poster Sessions*, 2006, pp. 835–841.
- [39] S. Barrachina et al., "Statistical approaches to computer-assisted translation," *Comput. Linguistics*, vol. 35, no. 1, pp. 3–28, 2009.
- [40] D. Ortiz-Martínez, I. García-Varea, and F. Casacuberta, "Online learning for interactive statistical machine translation," in *2010 Hum. Lang. Technol. Conf. North Am. Chapter Assoc. Comput. Ling., NAACL HLT 2010*, Los Angeles, CA, USA, 2010, pp. 546–554.
- [41] T. Lutellier, H. V. Pham, L. Pang, Y. Li, M. Wei, and L. Tan, "CoCoNuT: Combining context-aware neural translation models using ensemble for program repair," in *Proc. 29th ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*, 2020, pp. 101–114.
- [42] N. Jiang, T. Lutellier, and L. Tan, "CURE: Code-aware neural machine translation for automatic program repair," in *Proc. 43rd Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 1161–1173.
- [43] H. Ye, M. Martinez, and M. Monperrus, "Neural program repair with execution-based backpropagation," in *Proc. 44th Int. Conf. Softw. Eng. (ICSE)*, 2022, pp. 1506–1518.
- [44] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.
- [45] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [46] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Representations Poster (ICLR Poster)*, 2015.



Jiaqi Liu received the B.E. and Ph.D. degrees in electronic engineering from Shanghai Jiao Tong University, China, in 2014 and 2019 respectively. She is currently an Associate Professor with the School of Computer Science, Northwestern Polytechnical University. Her research interests include human-machine computing, social computing, and crowd sensing.



Fengming Zhang received the B.E. degree in network engineering from Nanjing Agricultural University, China, in 2022. He is currently working toward the master's degree with the Northwestern Polytechnical University. His research interests include human-machine computing and natural language processing.



Xin Zhang received the B.E. degree in engineering mechanics from China University of Mining & Technology, China, in 2020, and the master's degree from the Northwestern Polytechnical University, China, in 2023. His research interests include human-machine computing and natural language processing.



Zhiwen Yu (Senior Member, IEEE) received the Ph.D. degree of engineering in computer science and technology from the Northwestern Polytechnical University, in 2005. He is currently a Professor with the Northwestern Polytechnical University, China. He has worked as a Research Fellow with the Academic Center for Computing and Media Studies, Kyoto University, Japan, from 2007 to 2009, and a Postdoctoral Researcher with the Information Technology Center, Nagoya University, Japan, from 2006 to 2007. He has been an Alexander von Humboldt Fellow with Mannheim University, Germany, from 2009 to 2010. His research interests include pervasive computing, context-aware systems, human-computer interaction, mobile social networks, and personalization.



Liang Wang (Member, IEEE) received the Ph.D. degree from Shenyang Institute of Automation (SIA), Chinese Academy of Sciences, Shenyang, China, in 2014. He is currently a Professor with the Northwestern Polytechnical University, Xi'an, China. His research interests include ubiquitous computing, mobile crowd sensing, and Internet of Things.



Yao Zhang received the Ph.D. degree in telecommunication engineering from Xidian University, in 2020. He joined as a Postdoctoral Researcher with the School of Computer Science, Northwestern Polytechnical University in 2021, where he is now an Associated Professor. He was a Research Assistant and a Postdoctoral Fellow with The Hong Kong Polytechnic University in 2019 and 2021. His research interests include edge intelligence, crowd-sensing, and networked autonomous driving.



Bin Guo (Senior Member, IEEE) received the Ph.D. degree in computer science from Keio University, Japan, in 2009. He was a Postdoctoral Researcher with the Institut Telecom SudParis, France. He is a Professor with the Northwestern Polytechnical University, China. His research interests include ubiquitous computing, mobile crowd sensing, and HCI. He has served as an Associate Editor of *IEEE Communications Magazine* and *IEEE TRANSACTIONS ON HUMAN-MACHINE-SYSTEMS*, the Guest Editor of *ACM Transactions on Intelligent Systems and Technology* and *IEEE INTERNET OF THINGS*, the General Co-Chair of IEEE UIC'15, and the Program Chair of IEEE CPSCoM'16, ANT'14, and UIC'13.